

CSE 232 : Data Structures

LAB 02 : VECTOR

Due Saturday at 5:00 p.m. MST

The second programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) vector class. As you may recall, the `std::vector` class behaves much like the built-in array except it can grow to accommodate as many elements as the client demands. This lab will be conducted using the pair programming technique. Two or three students will work together to complete this lab.

Vector

Your vector class will need to be defined in its own header file (`vector.h`). The class name must be `vector` (or, more accurately, `custom::vector`). It will need to support the following operations:

- **Construct:** default constructor, copy constructor, move constructor, non-default empty fill constructor, non-default value fill constructor, and initializer list constructor.
- **Assign:** copy assign and move assign.
- **Iterator:** begin and end.
- **Access:** subscript operator, at method, front, and back. Each of these has both the read-only and read/write versions.
- **Insert:** push-back (both copy and move versions), reserve, and resize.
- **Remove:** pop-back, shrink to fit, and clear.
- **Status:** size, capacity, and empty.

The `vector` class has two methods which might need some explaining. As mentioned in the reading, the `resize()` method expands the size of the container to a given size. If an object had ten strings, then `resize(12)` will allocate a new buffer of size 12, copy over the initial 10, and add two empty strings onto the end. The `reserve()` method works similarly, except the new elements on the end are not initialized. In other words, `resize()` will change both the size (number of elements currently used) and capacity (number of elements allocated but not initialized) whereas the `reserve()` method will only change the capacity.

Iterator

Additionally, we will be creating a vector iterator. Only a read/write forward iterator is needed. This will exist in the same `vector.h`. This class will have a pointer to an vector element as the single member variable. The following methods will be supported:

- **Construct:** default constructor, copy constructor, and two non-default constructors. The first non-default constructor takes a pointer to a location in the vector data so `begin()` and `end()` can properly initialize it. The second takes an index and the vector.
- **Assign:** copy assign.
- **Compare:** equal-to and not-equal-to.
- **Access:** dereference (read-only and read/write).
- **Arithmetic:** increment (prefix and postfix), decrement (also prefix and postfix).

Three Options

As with last week, you will have three options when implementing this class.

85 Option : Integer Base Type

This option will require you to implement all the above interfaces, but integer will be the base data-type and no allocator will be required. This option will have three member variables:

```
int *    data;           // user data, a dynamically-allocated array
size_t  numCapacity;    // the capacity of the array
size_t  numElements;    // the number of items currently used
```

To complete the 85 option, please start with the following files: [232.02.Lab.85.zip](#).

100 Option : Template Base Type

This option will require you to implement all the same interfaces, but a template will be the base type and no allocator will be required.

```
template <typename T>
```

There are three member variables:

```
T *    data;           // user data, a dynamically-allocated array
size_t numCapacity;    // the capacity of the array
size_t numElements;    // the number of items currently used
```

To complete the 100 option, please start with the following files: [232.02.Lab.100.zip](#).

115 Option : Allocator

The 115 option will require the class to support an allocator.

```
template <typename T, typename A = std::allocator<T>>
```

There are four member variables:

```
A    alloc;           // use allocator for memory allocation
T *  data;           // user data, a dynamically-allocated array
size_t numCapacity;    // the capacity of the array
size_t numElements;    // the number of items currently used
```

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

To get full credit, you will also be required to get correct the precise number of allocations, frees, assignments, etc. To complete the 115 option, please start with the following files: [232.02.Lab.115.zip](#).

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.

2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `vector.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or no file was submitted.
Template Basetype 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Allocator 15%	115 solution was attempted				The 85 or the 100 solution was attempted